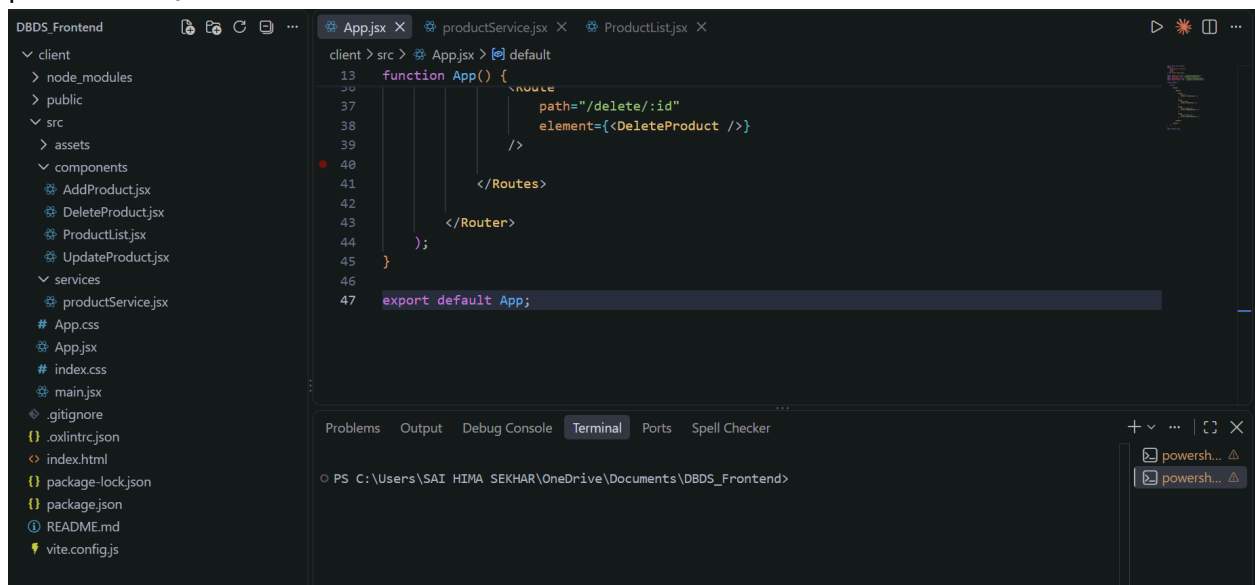
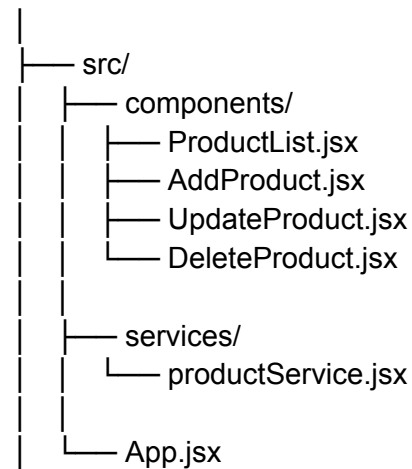


## REACTJS + SPRING BOOT CRUD

frontend/



ProductList.jsx

# REACTJS + SPRING BOOT CRUD

```
DBDS_Frontend
├── client
│   ├── node_modules
│   ├── public
│   └── src
│       ├── assets
│       ├── components
│       │   ├── AddProduct.jsx
│       │   ├── DeleteProduct.jsx
│       │   ├── ProductList.jsx
│       │   ├── UpdateProduct.jsx
│       └── services
│           ├── productService.jsx
│           ├── App.css
│           ├── App.jsx
│           ├── index.css
│           ├── main.jsx
│           ├── vite.config.js
│           ├── package-lock.json
│           ├── package.json
│           └── README.md
└── vite.config.js

App.jsx
productService.jsx
ProductList.jsx

client > src > components > ProductList.jsx > @ default
1 import React, { useEffect, useState } from "react";
2 import { getAllProducts, deleteProduct } from "../services/productService";
3
4 const ProductList = () => {
5
6     const [products, setProducts] = useState([]);
7
8     useEffect(() => {
9
10         getAllProducts()
11             .then((response) => {
12                 console.log(response.data);
13                 setProducts(response.data);
14             })
15             .catch((error) => {
16                 console.error("Error fetching products:", error);
17             });
18     }, []);
19
20     const handleDelete = (id) => {
21         deleteProduct(id)
22             .then(() => {
23                 setProducts(
24                     products.filter(product => product.id !== id)
25                 );
26             })
27             .catch((error) => {
28                 console.error("Error deleting product:", error);
29             });
30     };
31
32     return (
33         <div>
34             <h2>Product List</h2>
35
36             {products.length === 0 ? (
37                 <p>No products found.</p>
38             ) : (
39                 <ul>
40                     {products.map((product) => (
41                         <li key={product.id}>
42                             {product.name} -
43                             {product.price} -
44                             Quantity: {product.quantity}
45
46                             <button
47                                 onClick={() => handleDelete(product.id)}
48                                 >
49                                 Delete
50                             </button>
51                         </li>
52                     ))}
53                 </ul>
54             )}
55         </div>
56     );
57 }
58
59 export default ProductList;
```

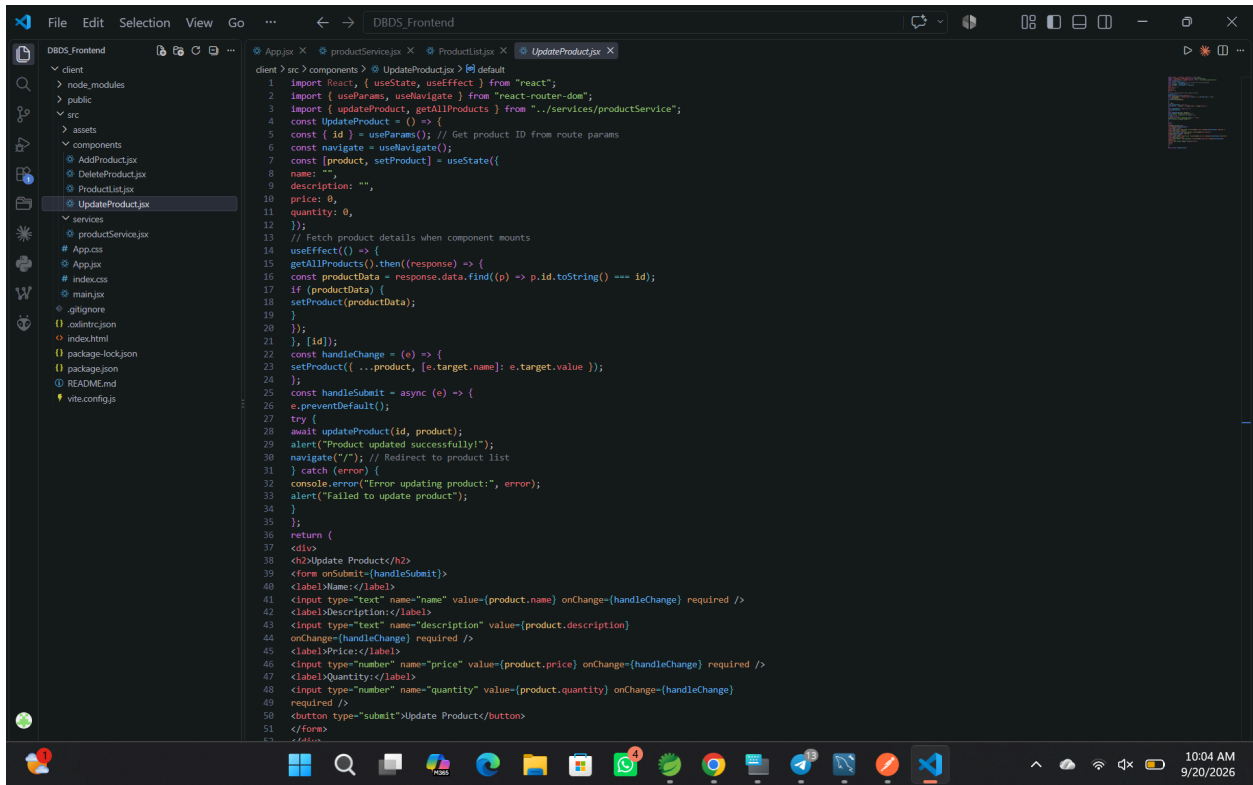
## AddProduct.jsx

```
File Edit Selection View Go ... DBDS_Frontend
App.jsx productService.jsx ProductList.jsx AddProduct.jsx

client > src > components > AddProduct.jsx > @ default
1 import React, { useState } from "react";
2 import { addProduct } from "../services/productService";
3
4 const AddProduct = () => {
5     const [product, setProduct] = useState({ name: "", description: "", price: "", quantity: "" });
6     const handleChange = (e) => {
7         setProduct((...product, { [e.target.name]: e.target.value }));
8     };
9     const handleSubmit = (e) => {
10         e.preventDefault();
11         addProduct(product).then(() => {
12             alert("Product added successfully!");
13         });
14     };
15     return (
16         <form onSubmit={handleSubmit}>
17             <input type="text" name="name" placeholder="Name" onChange={handleChange} required />
18             <input type="text" name="description" placeholder="Description" onChange={handleChange} />
19             <input type="number" name="price" placeholder="Price" onChange={handleChange} required />
20             <input type="number" name="quantity" placeholder="Quantity" onChange={handleChange} required />
21             <button type="submit">Add Product</button>
22         </form>
23     );
24 }
25
26 export default AddProduct;
```

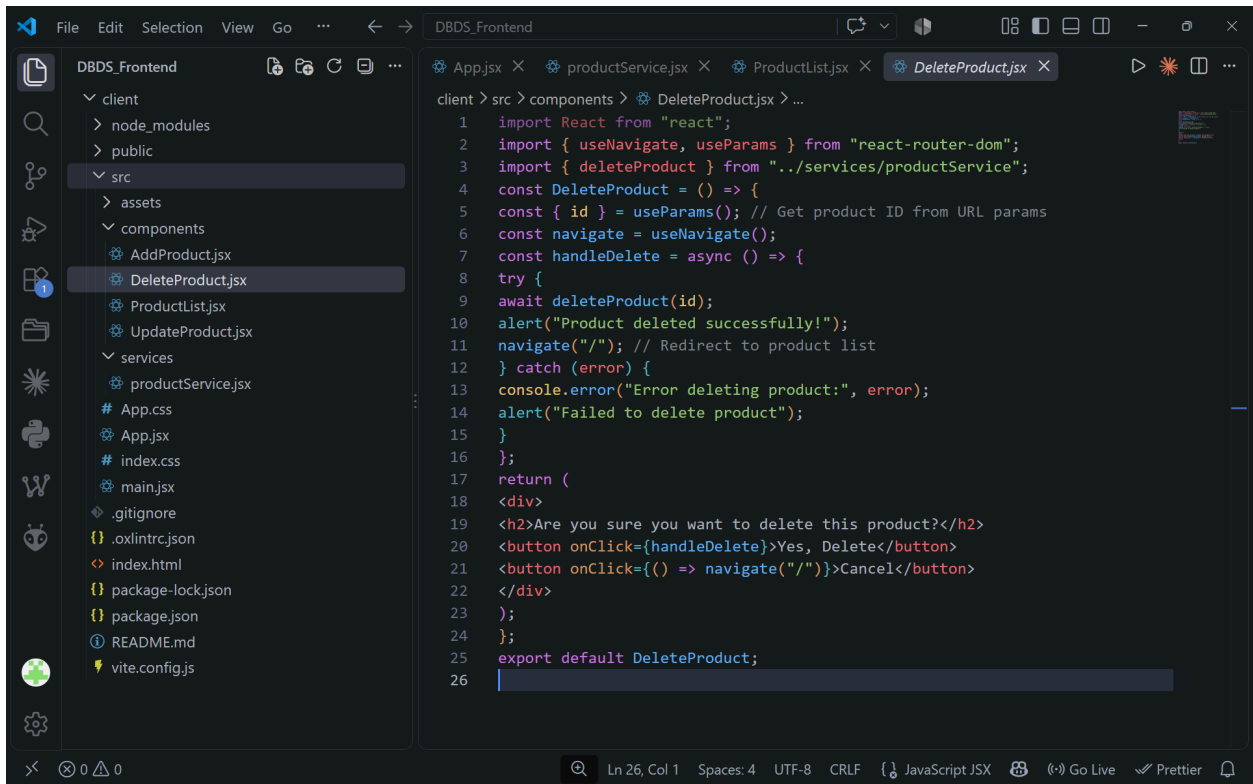
# REACTJS + SPRING BOOT CRUD

## UpdateProduct.jsx



```
client > src > components > UpdateProduct.jsx > default
1  import React, { useState, useEffect } from "react";
2  import { useParams, useNavigate } from "react-router-dom";
3  import { updateProduct, getAllProducts } from "../services/productService";
4  const UpdateProduct = () => {
5    const { id } = useParams(); // Get product ID from route params
6    const navigate = useNavigate();
7    const [product, setProduct] = useState({
8      name: "",
9      description: "",
10     price: 0,
11     quantity: 0,
12   });
13   // Fetch product details when component mounts
14   useEffect(() => {
15     getAllProducts().then((response) => {
16       const productData = response.data.find((p) => p.id.toString() === id);
17       if (productData) {
18         setProduct(productData);
19       }
20     });
21   }, [id]);
22   const handleChange = (e) => {
23     setProduct({ ...product, [e.target.name]: e.target.value });
24   };
25   const handleSubmit = async (e) => {
26     e.preventDefault();
27     try {
28       await updateProduct(id, product);
29       alert("Product updated successfully!");
30       navigate("/"); // Redirect to product list
31     } catch (error) {
32       console.error("Error updating product:", error);
33       alert("Failed to update product");
34     }
35   };
36   return (
37     <div>
38       <h2>Update Product</h2>
39       <form onSubmit={handleSubmit}>
40         <label>Name</label>
41         <input type="text" name="name" value={product.name} onChange={handleChange} required />
42         <label>Description</label>
43         <input type="text" name="description" value={product.description}
44           onChange={handleChange} required />
45         <label>Price</label>
46         <input type="number" name="price" value={product.price} onChange={handleChange} required />
47         <label>Quantity</label>
48         <input type="number" name="quantity" value={product.quantity} onChange={handleChange}
49           required />
50         <button type="submit">Update Product</button>
51       </form>
52     </div>
53   );
54 }
```

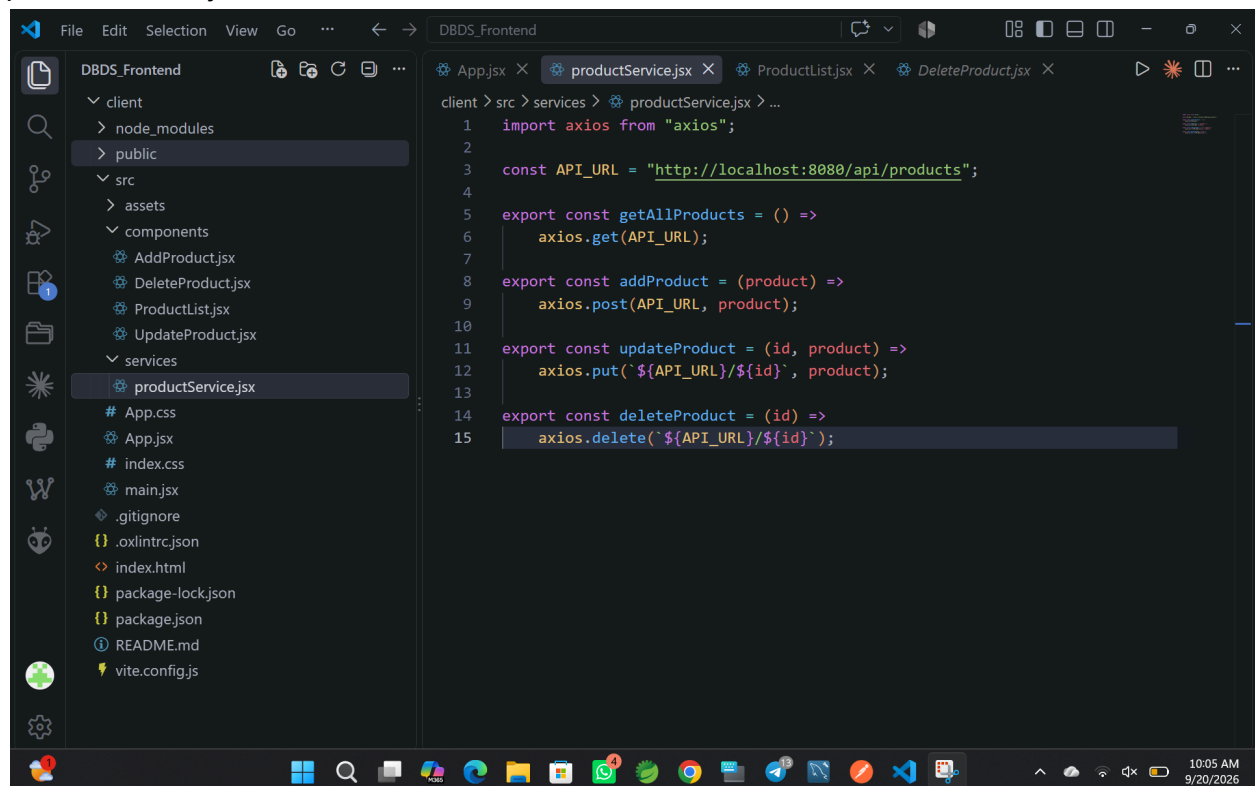
## DeleteProduct.jsx



```
client > src > components > DeleteProduct.jsx > ...
1  import React from "react";
2  import { useNavigate, useParams } from "react-router-dom";
3  import { deleteProduct } from "../services/productService";
4  const DeleteProduct = () => {
5    const { id } = useParams(); // Get product ID from URL params
6    const navigate = useNavigate();
7    const handleDelete = async () => {
8      try {
9        await deleteProduct(id);
10       alert("Product deleted successfully!");
11       navigate("/"); // Redirect to product list
12     } catch (error) {
13       console.error("Error deleting product:", error);
14       alert("Failed to delete product");
15     }
16   };
17   return (
18     <div>
19       <h2>Are you sure you want to delete this product?</h2>
20       <button onClick={handleDelete}>Yes, Delete</button>
21       <button onClick={() => navigate("/")}>Cancel</button>
22     </div>
23   );
24 };
25 export default DeleteProduct;
26
```

# REACTJS + SPRING BOOT CRUD

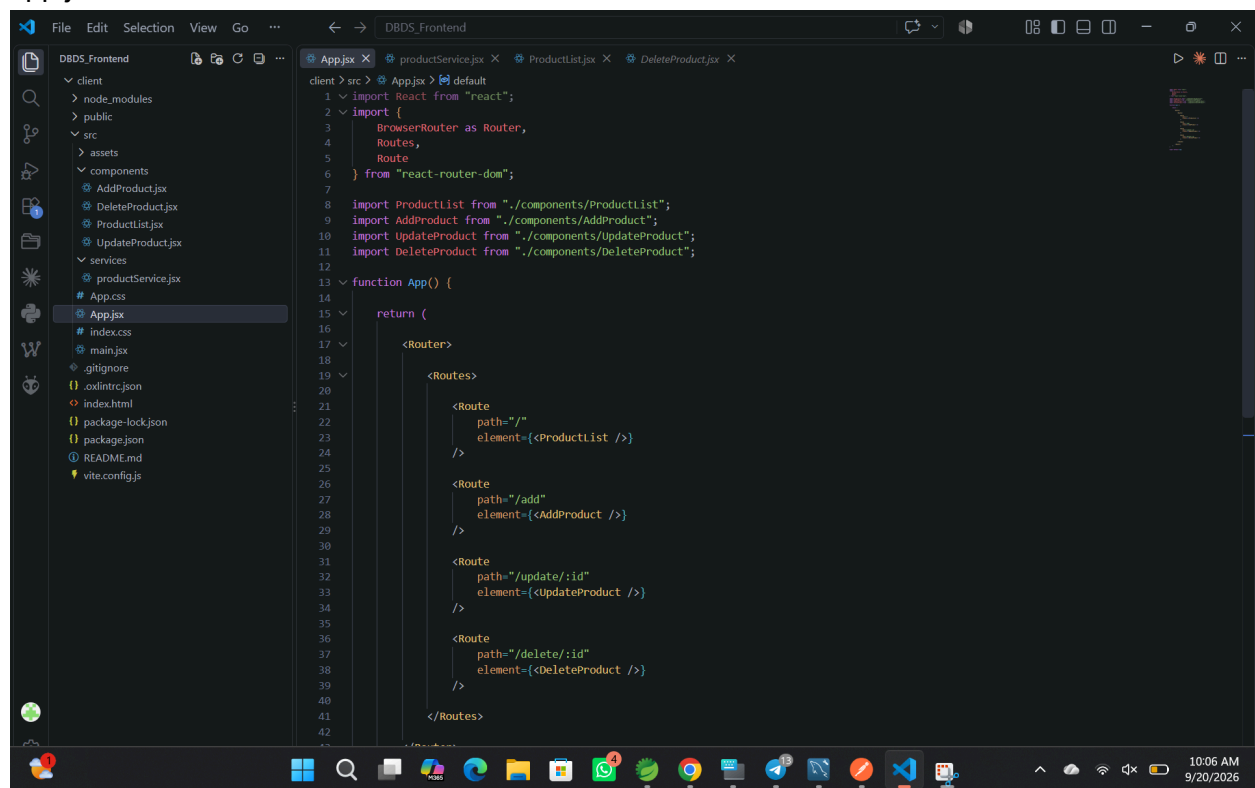
## productService.jsx



The screenshot shows a VS Code editor window with the file explorer on the left and the code editor on the right. The file explorer shows the project structure for DBDS\_Frontend, with the file productService.jsx selected under the services directory. The code editor displays the following JavaScript code:

```
client > src > services > productService.jsx > ...
1  import axios from "axios";
2
3  const API_URL = "http://localhost:8080/api/products";
4
5  export const getAllProducts = () =>
6  |   axios.get(API_URL);
7
8  export const addProduct = (product) =>
9  |   axios.post(API_URL, product);
10
11 export const updateProduct = (id, product) =>
12 |   axios.put(`${API_URL}/${id}`, product);
13
14 export const deleteProduct = (id) =>
15 |   axios.delete(`${API_URL}/${id}`);
```

## App.jsx:

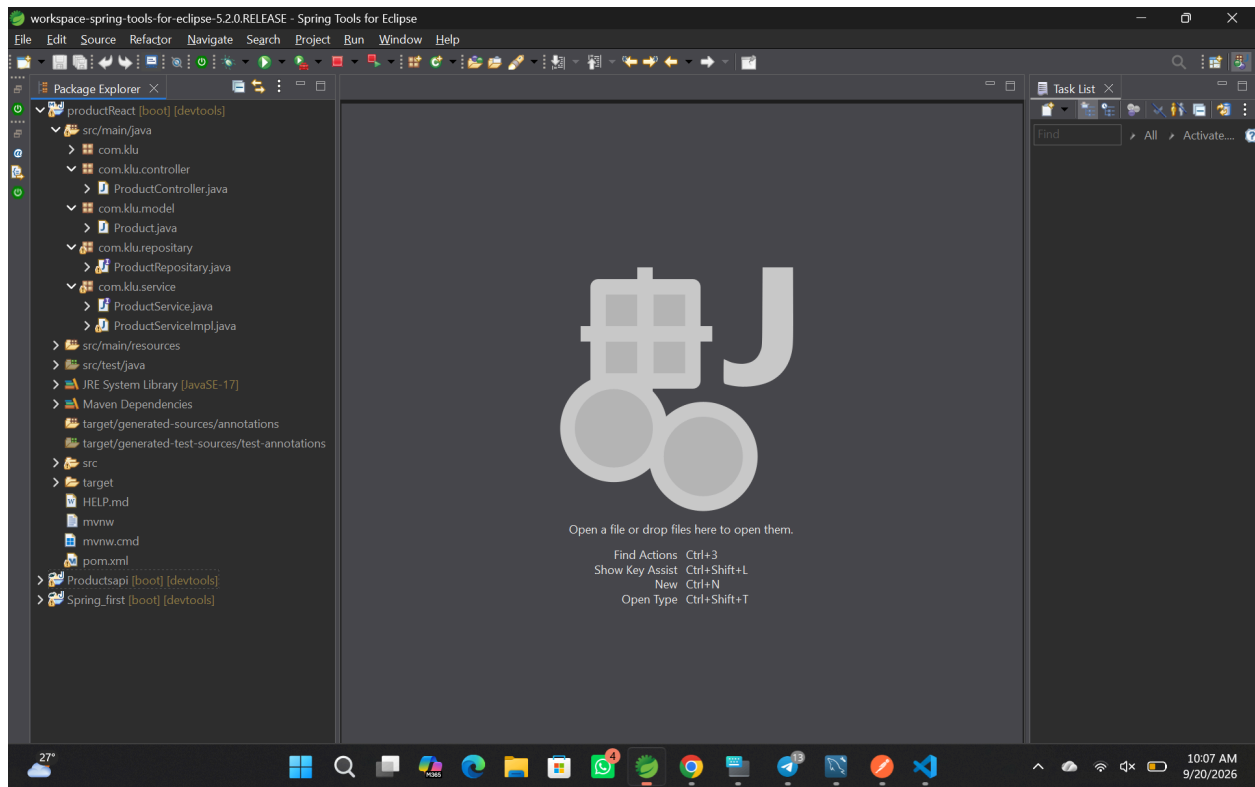


The screenshot shows a VS Code editor window with the file explorer on the left and the code editor on the right. The file explorer shows the project structure for DBDS\_Frontend, with the file App.jsx selected under the components directory. The code editor displays the following JavaScript code:

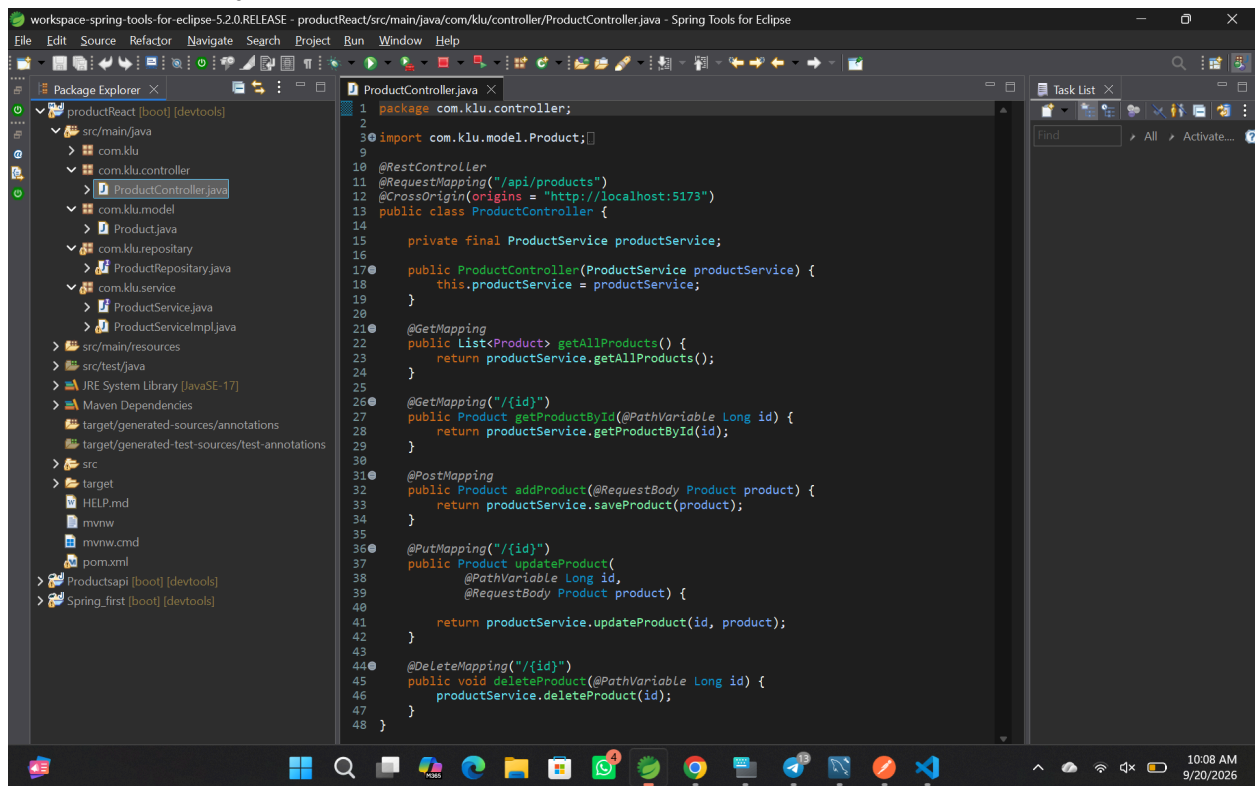
```
client > src > App.jsx > default
1  import React from "react";
2  import {
3    BrowserRouter as Router,
4    Routes,
5    Route
6  } from "react-router-dom";
7
8  import ProductList from "../components/ProductList";
9  import AddProduct from "../components/AddProduct";
10 import UpdateProduct from "../components/UpdateProduct";
11 import DeleteProduct from "../components/DeleteProduct";
12
13 function App() {
14
15   return (
16     <Router>
17       <Routes>
18         <Route
19           path="/"
20           element={<ProductList />}
21         />
22         <Route
23           path="/add"
24           element={<AddProduct />}
25         />
26         <Route
27           path="/update/:id"
28           element={<UpdateProduct />}
29         />
30         <Route
31           path="/delete/:id"
32           element={<DeleteProduct />}
33         />
34       </Routes>
35     </Router>
36   );
37 }
38
39 export default App;
```

# REACTJS + SPRING BOOT CRUD

## Backend(SpringBoot)

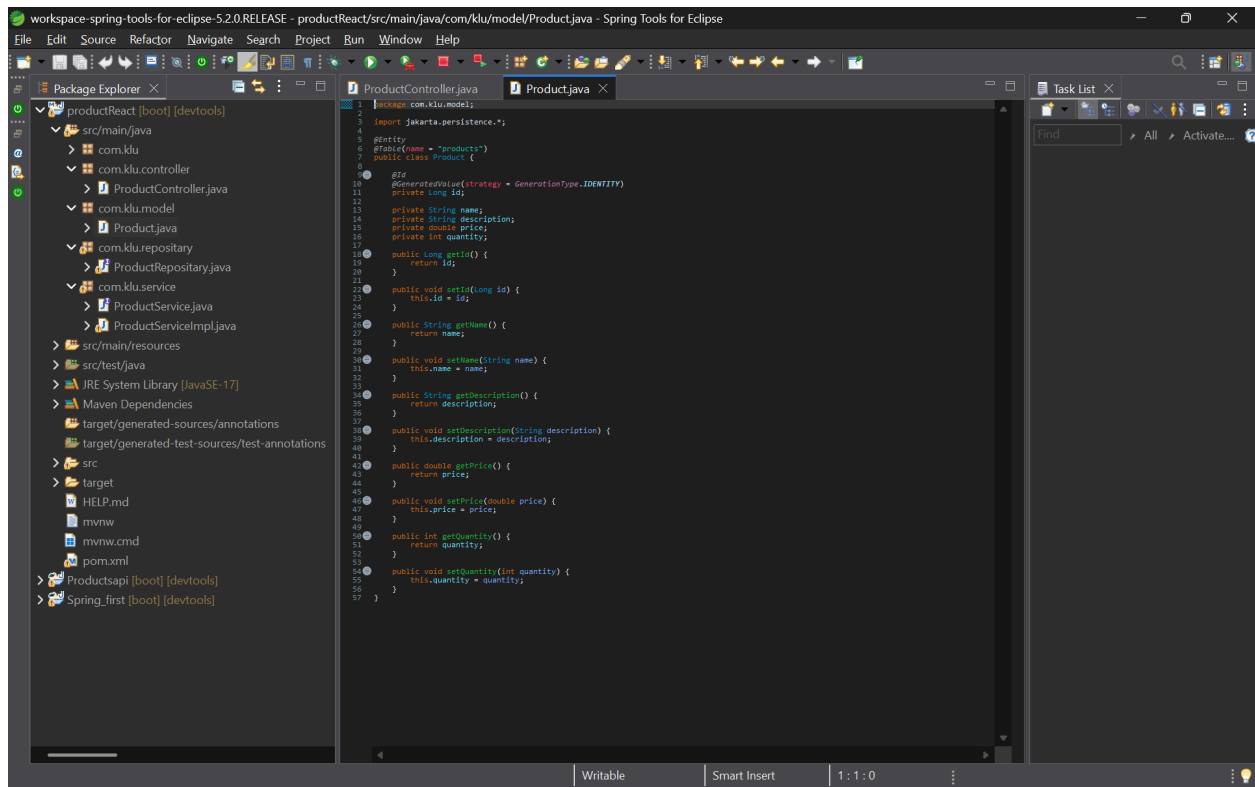


## ProductController.java



# REACTJS + SPRING BOOT CRUD

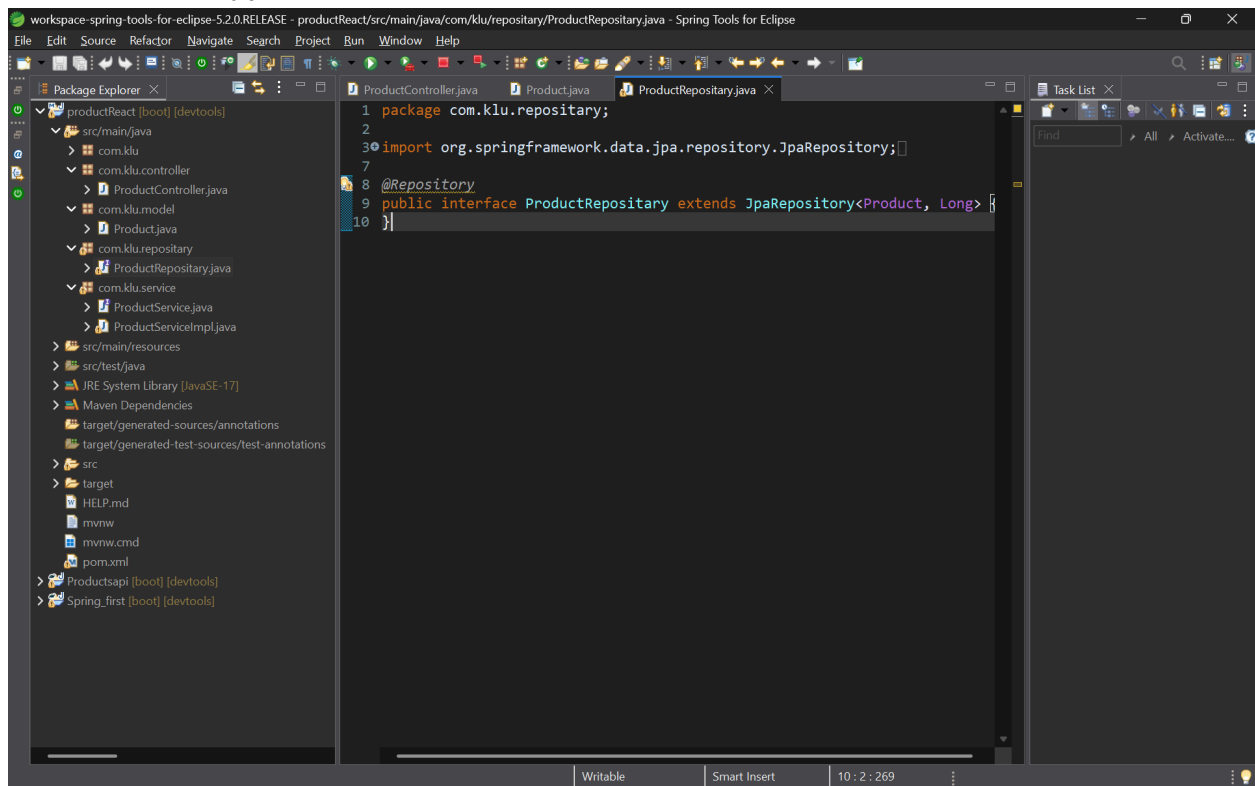
## Product.java



The screenshot shows the Eclipse IDE with the 'Product.java' file open. The Package Explorer on the left shows the project structure: productReact [boot] [devtools] > src/main/java > com.klu > com.klu.model > Product.java. The main editor displays the following code:

```
1 package com.klu.model;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 @Table(name = "products")
7 public class Product {
8
9     // @GeneratedValue(strategy = GenerationType.IDENTITY)
10     private Long id;
11
12     private String name;
13     private String description;
14     private double price;
15     private int quantity;
16
17     public Long getId() {
18         return id;
19     }
20
21     public void setId(Long id) {
22         this.id = id;
23     }
24
25     public String getName() {
26         return name;
27     }
28
29     public void setName(String name) {
30         this.name = name;
31     }
32
33     public String getDescription() {
34         return description;
35     }
36
37     public void setDescription(String description) {
38         this.description = description;
39     }
40
41     public double getPrice() {
42         return price;
43     }
44
45     public void setPrice(double price) {
46         this.price = price;
47     }
48
49     public int getQuantity() {
50         return quantity;
51     }
52
53     public void setQuantity(int quantity) {
54         this.quantity = quantity;
55     }
56
57 }
```

## ProductRepository.java

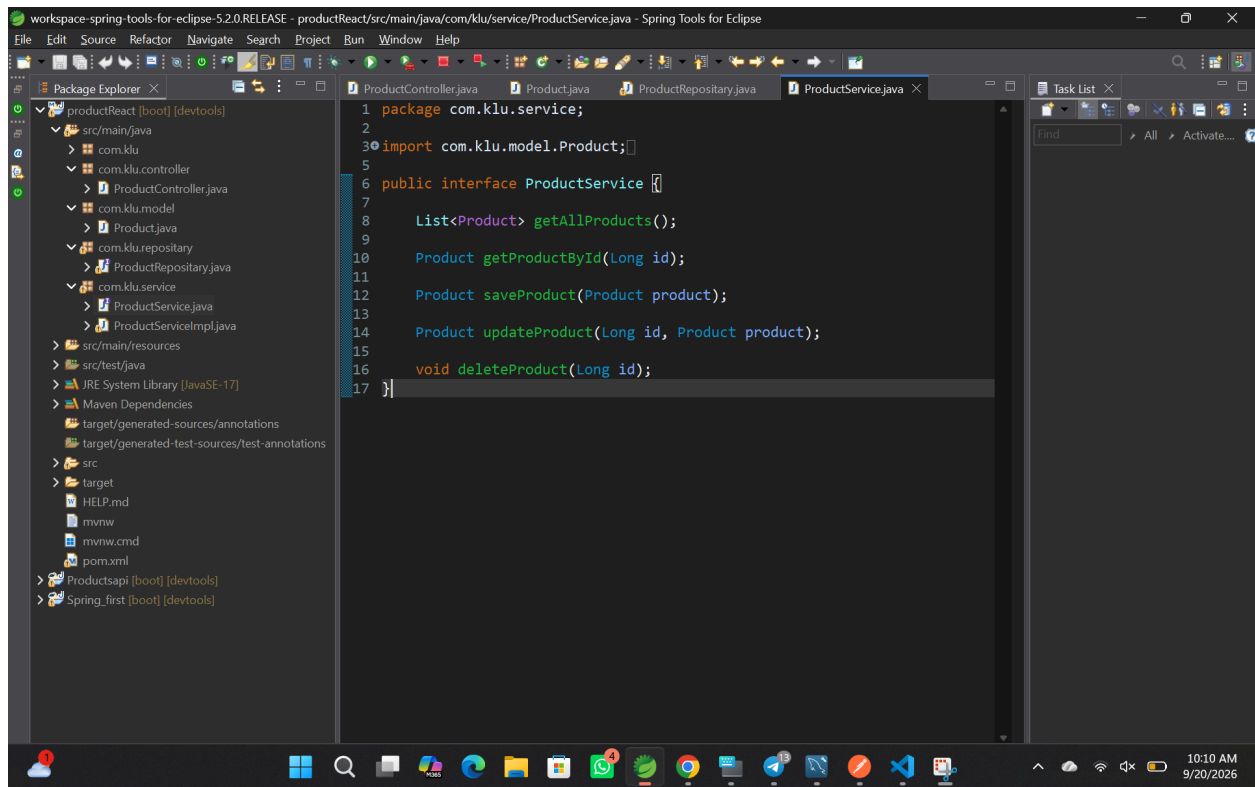


The screenshot shows the Eclipse IDE with the 'ProductRepository.java' file open. The Package Explorer on the left shows the project structure: productReact [boot] [devtools] > src/main/java > com.klu > com.klu.repository > ProductRepository.java. The main editor displays the following code:

```
1 package com.klu.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 @Repository
6 public interface ProductRepository extends JpaRepository<Product, Long> {
7
8 }
```

# REACTJS + SPRING BOOT CRUD

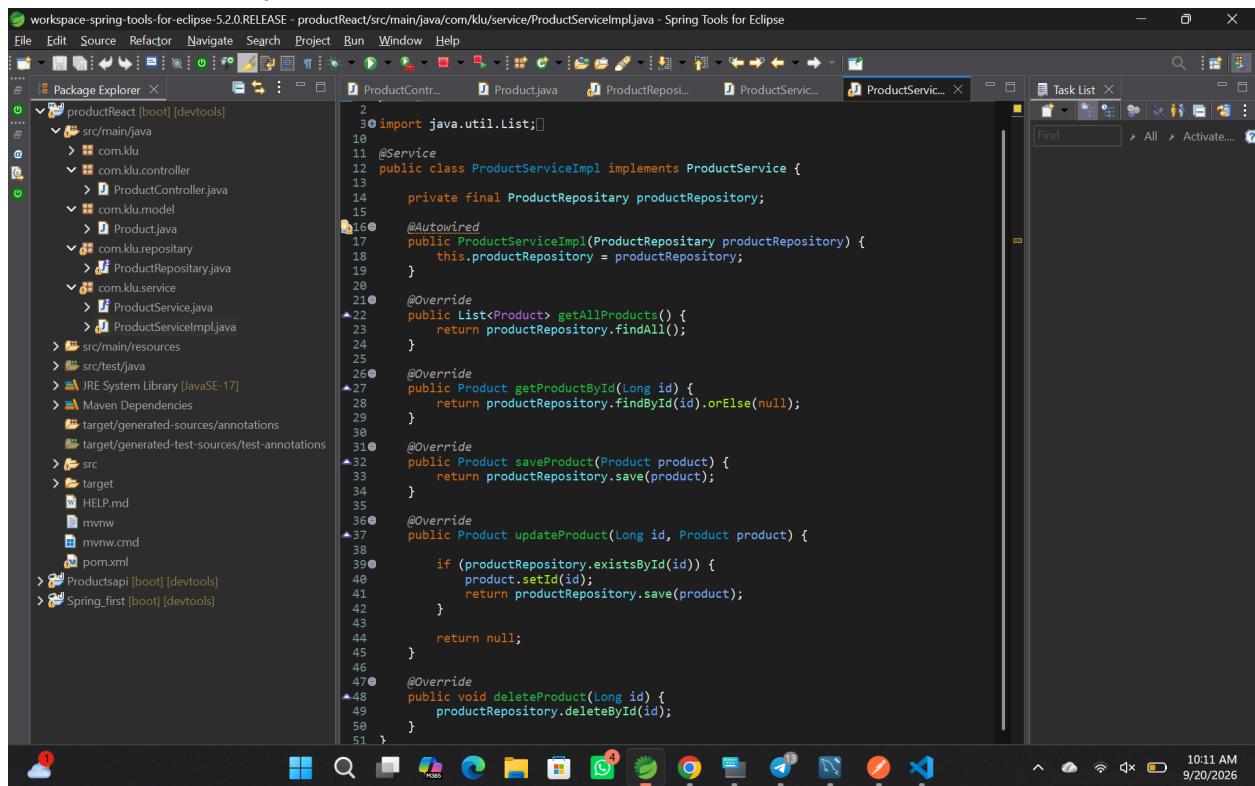
## ProductService.java



The screenshot shows the Eclipse IDE with the 'ProductService.java' file open. The Package Explorer on the left shows the project structure: 'productReact [boot] [devtools]' containing 'src/main/java' with packages 'com.klu', 'com.klu.controller', 'com.klu.model', 'com.klu.repository', and 'com.klu.service'. The 'com.klu.service' package contains 'ProductService.java' and 'ProductServiceImpl.java'. The main editor displays the code for 'ProductService.java'.

```
1 package com.klu.service;
2
3 import com.klu.model.Product;
4
5
6 public interface ProductService {
7
8     List<Product> getAllProducts();
9
10    Product getProductById(Long id);
11
12    Product saveProduct(Product product);
13
14    Product updateProduct(Long id, Product product);
15
16    void deleteProduct(Long id);
17 }
```

## ProductServiceImpl.java

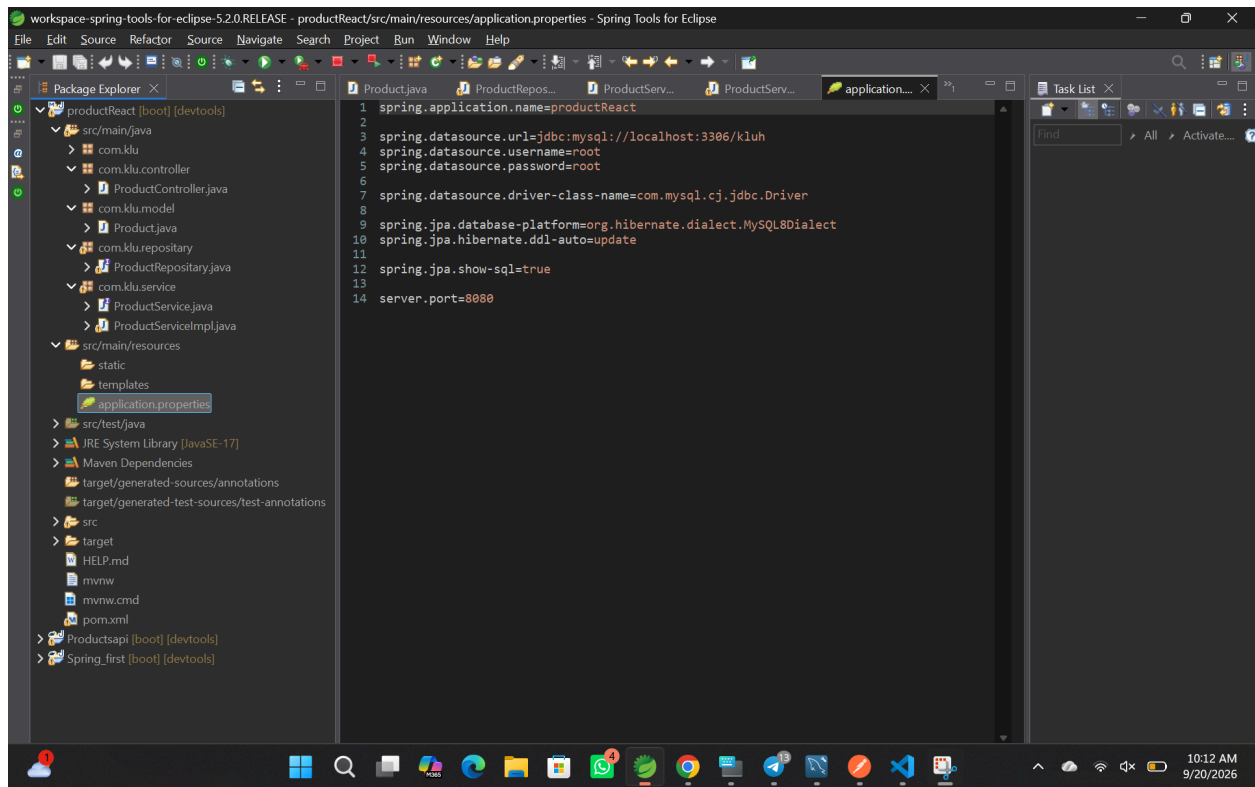


The screenshot shows the Eclipse IDE with the 'ProductServiceImpl.java' file open. The Package Explorer on the left shows the same project structure as the previous screenshot. The main editor displays the code for 'ProductServiceImpl.java'.

```
1
2
3 import java.util.List;
4
5
6 @Service
7 public class ProductServiceImpl implements ProductService {
8
9     private final ProductRepository productRepository;
10
11     @Autowired
12     public ProductServiceImpl(ProductRepository productRepository) {
13         this.productRepository = productRepository;
14     }
15
16     @Override
17     public List<Product> getAllProducts() {
18         return productRepository.findAll();
19     }
20
21     @Override
22     public Product getProductById(Long id) {
23         return productRepository.findById(id).orElse(null);
24     }
25
26     @Override
27     public Product saveProduct(Product product) {
28         return productRepository.save(product);
29     }
30
31     @Override
32     public Product updateProduct(Long id, Product product) {
33         if (productRepository.existsById(id)) {
34             product.setId(id);
35             return productRepository.save(product);
36         }
37         return null;
38     }
39
40     @Override
41     public void deleteProduct(Long id) {
42         productRepository.deleteById(id);
43     }
44 }
```

# REACTJS + SPRING BOOT CRUD

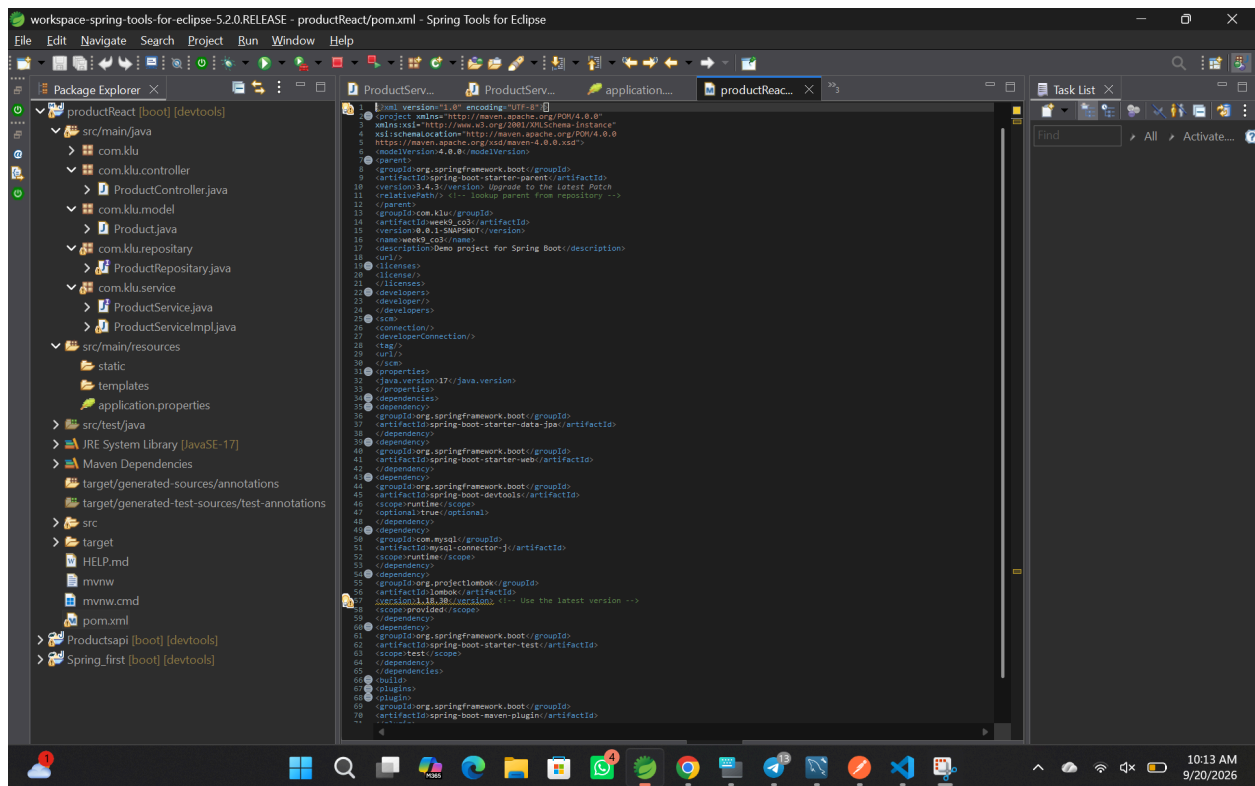
application . properties:



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure for 'productReact'. The main editor window shows the 'application.properties' file with the following content:

```
1 spring.application.name=productReact
2
3 spring.datasource.url=jdbc:mysql://localhost:3306/kluh
4 spring.datasource.username=root
5 spring.datasource.password=root
6
7 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
8
9 spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
10 spring.jpa.hibernate.ddl-auto=update
11
12 spring.jpa.show-sql=true
13
14 server.port=8080
```

pom.xml:



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure for 'productReact'. The main editor window shows the 'pom.xml' file with the following content:

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5         http://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7     <groupId>org.springframework.boot</groupId>
8     <artifactId>spring-boot-starter-parent</artifactId>
9     <version>3.4.2</version>
10    <relativePath>..</relativePath>
11    <parent>
12        <groupId>com.klu</groupId>
13        <artifactId>week9_c03</artifactId>
14        <version>0.0.1-SNAPSHOT</version>
15        <name>week9_c03</name>
16        <description>Demo project for Spring Boot</description>
17        <url></url>
18        <licenses>
19            <license>
20                <name></name>
21                <url></url>
22            </license>
23            <license>
24                <name></name>
25                <url></url>
26            </license>
27        </licenses>
28        <developers>
29            <developer>
30                <name></name>
31                <email></email>
32                <organization></organization>
33            </developer>
34        </developers>
35        <scm>
36            <connection></connection>
37            <developerConnection></developerConnection>
38            <tag></tag>
39            <url></url>
40        </scm>
41        <properties>
42            <java.version>17</java.version>
43        </properties>
44        <dependencies>
45            <dependency>
46                <groupId>org.springframework.boot</groupId>
47                <artifactId>spring-boot-starter-data-jpa</artifactId>
48                <scope></scope>
49            </dependency>
50            <dependency>
51                <groupId>org.springframework.boot</groupId>
52                <artifactId>spring-boot-starter-web</artifactId>
53                <scope></scope>
54            </dependency>
55            <dependency>
56                <groupId>org.springframework.boot</groupId>
57                <artifactId>spring-boot-devtools</artifactId>
58                <scope>runtime</scope>
59                <optional>true</optional>
60            </dependency>
61            <dependency>
62                <groupId>com.mysql</groupId>
63                <artifactId>mysql-connector-j</artifactId>
64                <scope>runtime</scope>
65            </dependency>
66            <dependency>
67                <groupId>org.projectlombok</groupId>
68                <artifactId>lombok</artifactId>
69                <scope>provided</scope>
70            </dependency>
71            <dependency>
72                <groupId>org.springframework.boot</groupId>
73                <artifactId>spring-boot-starter-test</artifactId>
74                <scope>test</scope>
75            </dependency>
76        </dependencies>
77        <build>
78            <plugins>
79                <plugin>
80                    <groupId>org.springframework.boot</groupId>
81                    <artifactId>spring-boot-maven-plugin</artifactId>
82                </plugin>
83            </plugins>
84        </build>
85    </project>
```

## REACTJS + SPRING BOOT CRUD

OUTPUT:

